I. Format d'une instruction :

Chaque instruction est représentée par une ligne de codes. En assembleur, une instruction est divisée en champs.

Etiquette : Code opération (mnemonique) Opérande(s) ; commentaire

- le code opération indique au processeur quelle instruction réaliser ;
- le champ opérande contient la donnée, ou la référence à une donnée en mémoire (son adresse).

II. Instructions de transfert

Elle permet de transférer les données (un octet ou un mot) , sa syntaxe est comme suit :

MOV destination, source

L'instruction peut etre de la forme :

MOV Reg/Var, Reg
MOV Reg, Reg/Var
MOV Reg/Var, const

Exemples :

```
MOV   AX, BX    ;  Transfert d'un registre de 16 bits vers un registre de 16 Bits
MOV   AH, CL    ;  Transfert d'un registre de 8 bits vers un registre de 8 bits
MOV   AX, 1234h  ; Transfert d'une constante vers AX
```

Remarque 1 :
Il n'est pas possible de transférer le contenu d'une case mémoire vers une autre case mémoire
Remarque 2 : MOV AL, BX ; BX → AL (erreur!! → taille des registres)
Remarque 3 : MOV [DI], AH (copie un octet) et MOV [SI], AX (copie un mot avec convention little Endian pour les microprocesseurs Intel comme le 8086)

III. Instructions arithmétiques

A. Addition :

Syntaxe : ***ADD Destination, source***

Elle permet d'additionner le contenu de la source (octet ou un mot) avec celui de la destination le résultat est mis dans la destination Destination <---------- Destination + source

L'instruction peut être de la forme :

ADD Reg/Var, Reg
ADD Reg, Reg/Var
ADD Reg/Var, const

Exemples :

```
ADD AX, BX  ; AX = AX + BX (addition sur 16 bits) ADD  AL,BH
        ; AL = AL + BH (addition sur 8 bits )
ADD AL, [SI]  ; AL = AL + le contenu de la case mémoire pointé par SI
ADD [DI], AL ; le contenu de la case mémoire pointé par DI est additionnée avec AL, le
                résultat est mis dans la case mémoire pointé par DI
```

Remarque :
On n'a pas le droit d'additionner deux cases mémoires sans utiliser un registre de données.

B. SUB : (Soustraction)

Syntaxe : ***SUB Destination, source***
Elle permet de soustraire la destination de la source (octet ou un mot) le résultat est mis dans la destination Destination <--------- Destination -- source

C. INC : (Incrémentation)

Syntaxe : ***INC Destination***
Elle permet d'incrémenter le contenu de la destination Destination <---------- Destination + 1
Exemples :
INC AX ; AX = AX + 1 (incrémentation sur 16 bits).
INC AL ; AL = AL +1 (incrémentation sur 8 bits).
INC [SI] ; [SI] = [SI] + 1 le contenu de la case mémoire pointé par SI sera incrémenter

D. DEC : (Décrémentation)

Syntaxe : ***DEC Destination***
Elle permet de décrémenter le contenu de la destination Destination <---------- Destination - 1

E. MUL : (Multiplication)

Elle a la forme : ***MUL Reg/Var***
Dans sa première forme qui prend une donnée 8 bits en opérande (donc le résultat est sur 16 bits), l'instruction mul effectue le produit de la valeur contenue dans le registre al avec la valeur de l'opérande fourni. Le résultat est placé dans le registre ax.
Dans sa deuxième forme qui prend une donnée 16 bits en opérande (donc le résultat est sur 32 bits), l'instruction mul effectue le produit de la valeur contenue dans le registre ax avec la valeur de l'opérande fourni. Le résultat est placé dans la paire de registres dx et ax. dx contient le poids fort du résultat, ax le poids faible.
Donc on aura :
(AX) <---------- (AL) x Source (octet)
(AX)(DX) <---------- (AX) x Source (mot)
Exemples :

```
mov al, 4          mov bx, 435
mov ah, 25         mov ax, 2372
imul ah            imul bx
```

IV. Instructions logiques :

Les instructions logiques effectuent des opérations logiques bit à bit. On dispose de trois opérateurs logiques : ET, OU et OU exclusif. Il n'y a jamais propagation de retenue lors de ces opérations (chaque bit du résultat est calculé indépendamment des autres).

A. L'instruction AND (ET) : Elle a la forme
AND Reg/Var, Reg
AND Reg, Reg/Var
AND Reg/Var, const

AND est souvent utilisé pour forcer certains bits à 0. Après AND AX, FF00, l'octet de poids faible de AX vaut 00, tandis que l'octet de poids fort est inchangé.

B. L'instuction OR (OU) (même forme)
OR est souvent utilisé pour forcer certains bits à 1. Par exemple après OR AX, FF00, l'octet de poids fort de AX vaut FF, tandis que l'octet de poids faible est inchangé.

C. L'instruction XOR (OU exclusif): (même forme)
XOR est souvent utilisé pour inverser certains bits. Après XOR AX, FFFF, tous les bits de AX sont inversés.

Exemples :

	0 0 1 1	0 0 1 1	0 0 1 1
OU	0 1 0 1	ET 0 1 0 1	OU EX 0 1 0 1
	---	---	---
	0 1 1 1	0 0 0 1	0 1 1 0

V. Instruction de comparaison

CMP permet de comparer deux valeurs. C'est l'instruction la plus utilisée pour positionner les indicateurs avant d'effectuer une instruction de saut conditionnel. Sa forme :

CMP registre/variable, registre
CMP registre, registre/variable
CMP registre/variable, constante

CMP permet de comparer deux valeurs. Pour cela CMP soustrait le second opérande du premier, sans modifier l'opérande destination, mais en positionnant les indicateurs en fonction du résultat.

Exemple :
mov al, 10
cmp al, 12
le registre AL n'est pas modifié par l'exécution de l'instruction CMP et contient toujours la valeur 10. L'indicateur ZF est égal à 0, cela indique que les deux données sont différentes. L'indicateur CF est positionné à 1, ce qui indique que le deuxième opérande est supérieur à la valeur du premier opérande. Le bit de signe S est également mis à 1 car 10 − 12 donne un nombre négatif.

VI. Instructions de branchement

La prochaine instruction à exécuter est repérée en mémoire par le registre IP. Les instructions de branchement permettent de modifier la valeur de IP pour exécuter une autre instruction (boucles, tests, etc.).
On distingue deux types de branchements :
– branchements inconditionnels : IP adresse d'une instruction ;
– branchements conditionnels : Si une condition est satisfaite, alors branchement, sinon passage simple à l'instruction suivante.

A. Sauts conditionnels

L'instruction JMP effectue un saut inconditionnel à l'étiquette spécifiée. Sa syntaxe est :
JMP étiquette

B. Sauts conditionnels

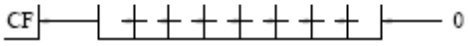
Les instructions de branchements conditionnels effectuent un saut (comme JMP) si une certaine condition est vérifiée. Si ce n'est pas le cas, le processeur passe à l'instruction suivante.
Les conditions s'expriment en fonction des valeurs des indicateurs. Les instructions de branchement conditionnel s'utilisent en général immédiatement après une instruction de comparaison CMP.

Voici la liste des instructions de branchement les plus utiles :
JE etiquette *Jump if Equal* saut si ZF = 1 ;
JNE etiquette *Jump if Not Equal* saut si ZF = 0 ;
JG etiquette *Jump if Greater* saut si ZF = 0 et SF = 0 ;
JL etiquette *Jump if Lower* saut si SF≠0
JLE etiquette *Jump if Lower or Equal* saut si ZF=1 ou SF≠0 ;

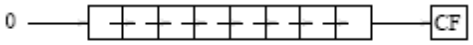
VII. Instructions de décalage et de rotation

Ces opérations décalent vers la gauche ou vers la droite les bits du registre.

SHL *registre,* 1 *(Shift Left)*
Décale les bits du registre d'une position vers la gauche. Le bit de gauche est transféré dans l'indicateur CF. Les bits introduits à droite sont à zéro.

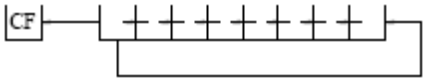


SHR *registre,* 1 *(Shift Right)*
Comme SHL mais vers la droite. Le bit de droite est transféré dans CF.

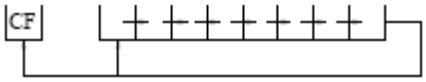


SHL et SHR peuvent être utilisé pour multiplier/diviser des entiers *naturels* (et non des relatifs car le bit de signe est perdu).

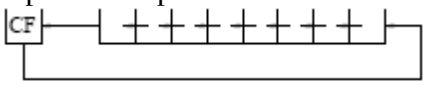
ROL *registre,* 1 *(Rotate Left)*
Rotation vers la gauche : le bit de poids fort passe à droite, et est aussi copié dans
CF. Les autres bits sont décalés d'une position.



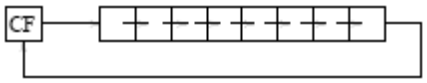
ROR *registre,* 1 *(Rotate Right)*
Comme ROL, mais à droite.



RCL *registre,* 1 *(Rotate Carry Left)*
Rotation vers la gauche en passant par l'indicateur CF. CF prend la place du bit de poids faible ; le bit de poids fort part dans CF.



RCR *registre*, 1 *(Rotate Carry Right)*
Comme RCL, mais vers la droite.



RCL et RCR sont utiles pour lire bit à bit le contenu d'un registre (On pourra utiliser l'instruction JB pour brancher si CF=1 après RCL ou RCR).

Leur syntaxe se ressemble. On prends l'exemple de l'instruction RCL

RCL registre/variable, 1
RCL registre/variable, CL

RCL effectue une rotation `a gauche de l'opérande destination indiqué, 1 ou CL fois.

VIII. Mode d'adressage

Le processeur peut adresser des opérandes (Données) de plusieurs manières, appelés Modes d'Adressage.

1. Adressage implicite

L'instruction contient seulement le code opération. L'instruction porte sur des registres ou spécifie une opération sans opérande (exemple : "incrémenter AX").
Ex assembleur : INC AX en langage machine : 40h

2. Adressage immédiat

Le champ opérande contient la donnée
Ex assembleur : MOV AX, 2550h ADD BL, 40h (sauf CS, DS, SS)

3. Adressage par Registre

Le champ opérande contient un registre
Ex assembleur : MOV DX, BX ADD AL, AH

4. Adressage direct

Le champ opérande contient l'*adresse* de la donnée en mémoire principale
Ex assembleur : MOV DL, [2550]

5. Adressage indirect (Basé):

L'adresse de la donnée est contenue dans le registre. On a deux registres de base BX et BP
Ex : MOV AX,[BX]
Remarque : implicitement c'est équivalent à MOV AX,DS:[BX] sinon il faut le spécifier comme pour le segment supplémentaire ES on aura MOV AX [ES :BX]
MOV AX,[BP] c'est équivalent à MOV AX,SS:[BP]

6. Adressage basé relatif (avec déplacement)

Ex : MOV [BX+100h],AL

7. Adressage indéxé (SI ou DI)

Ex : MOV [SI],AX

8. Adressage indéxé relatif (avec déplacement)

Ex : MOV [DI+100h],AL

9. Adressage basé indéxé
Ex : MOV AX,[BX+SI]

10. Adressage basé indéxé relatif
Ex : MOV AX,[BX+DI+100h]

11. Adressage relatif
Ce mode d'adressage est utilisé pour certaines instructions de branchement. Le champ opérande contient un entier relatif, nommé déplacement, qui sera ajouté à la valeur courante de IP.
Ex : JMP 200h

Remarque :
MOV [100h],34 est un adressage direct

IX Passage vers l'assembleur :
IX.1 Instruction de test conditionnel :
Un test dans un langage évolué peut avoir la forme suivante :

SI (condition vraie) ALORS
action-alors
SINON
action-sinon
FIN_SI

En assembleur, ce type de construction est réalisé à l'aide du programme suivant :

```
        calcul de la condition
        JmpCondit SINON   ; utilisation d'une instruction de branchement conditionnel
        action-alors
        ...
        JMP FSI
SINON: action-sinon
        ...
FSI :   ...
```

IX.2 Boucle Tantque :
La syntaxe d'une boucle Tantque, dans un langage de haut niveau, est le suivant :

TANTQUE (condition) FAIRE
action
FIN_TQ

Cela se traduit en assembleur sous la forme suivante :

```
TQ : calcul de la condition
        JmpCondit FTQ   ; utilisation d'une instruction de branchement conditionnel
        action
        ...
        JMP TQ
FTQ :   ...
```

IX.3 Boucle Répéter

La syntaxe d'une boucle REPETER dans un langage de haut niveau est le suivant :

REPETER
action
JUSQUA (condition vraie)

Cela se traduit de la manière suivante en assembleur :

```
REPETER :
        action
        ...
        calcul de la condition
        JmpCondit REPETER
```

IX.4 Boucle Pour :

La syntaxe d'une boucle Pour s'écrit de la manière suivante :
POUR indice := 1 A valeur FAIRE
action
FAIT

Cette boucle se traduit en assembleur de la manière suivante :

```
        mov cx, bs
POUR : ...
        action
        loop POUR
```

L'instruction LOOP fonctionne avec le registre CX qui joue le rôle de compteur de boucles. LOOP décrémente le compteur sans modifier aucun des indicateurs. Si le compteur est différent de 0, un saut à l'étiquette opérande de l'instruction LOOP est réalisé.

VI. Opérations d'affichage et de lecture :

Les fonctions du système s'appellent à l'aide du vecteur 21H. La valeur du registre AH permet d'indiquer quelle est la fonction que l'on appelle :

```
MOV AH, numero_fonction
INT 21H
```

La fonction 4CH du système permet de terminer un programme et de revenir au système.

Exemple de fonction :

Numéro	Fonction
01H	Lecture caractère. Met le code ascii lu dans AL
02H	Affiche caractère. Le code ascii dans registre DL

Le programme suivant lit un caractère au clavier et l'affiche en majuscule :

```
MOV AH, 01H
INT 21H
AND AL, 11011111b
MOV DL, AL
MOV AH, 02H
INT 21H
```